Inference with a PING network

Re-using the weights from an AI-SNN to run inference with PING.

Eoin Murray 2nd-March-2026

Abstract.

- Extremely quickly done and rough work, but I think (hope!) the principle holds.
- We start with an ANN classifying MNIST, bog standard approach, followed PyTorch tutorial, and achieve ***97.4%*** accuracy, as expected.
- Then convert the ANN into an standard SNN with surrogate gradient and BPTT, and achieve ***64.2%*** accuracy first attempt, with no attempt to improve.
- **Add inhibitory pool to the standard SNN hidden layer to make a PING-SNN.**
- Run inference *reusing* the weights from the standard-SNN training **with no PING-SNN training.**
- PING-SNN achieved ***55.3%*** showing clear PING rhythms in the hidden layer.

MNIST with an ANN: Setup

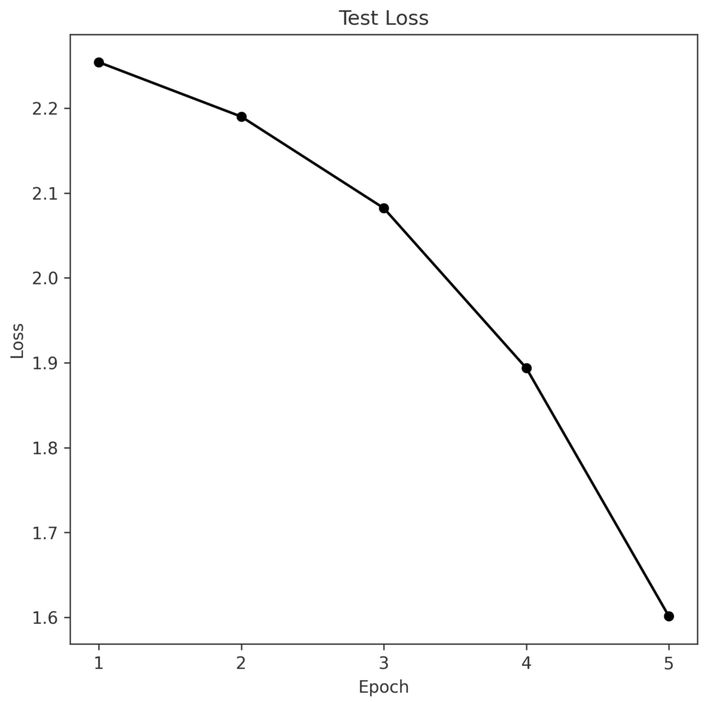
We start with the PyTorch tutorial

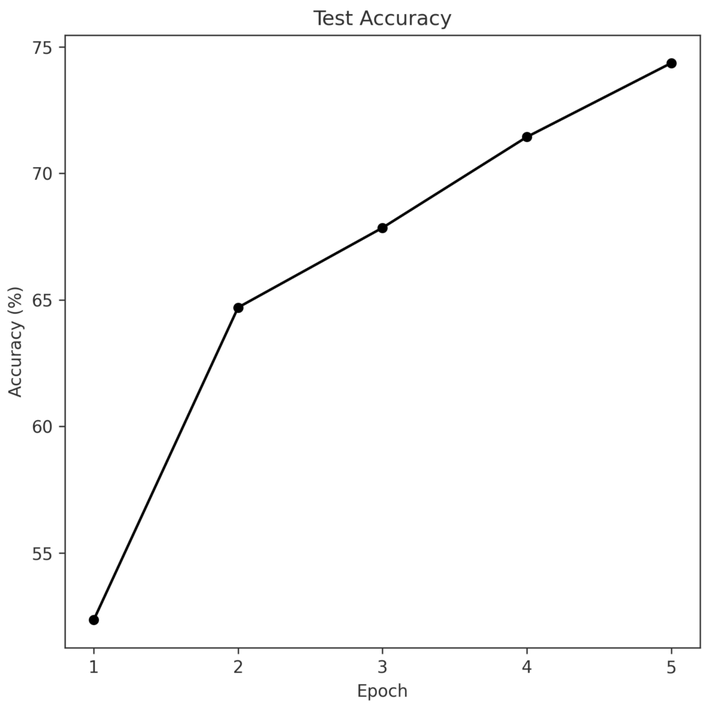
- Three-layer feedforward MLP.
- ReLU activations
- Adam optimizer
- 5 epochs, batch size 64
- Trained on full 60,000 images
- Evaluated on 10,000 images
- Lr 10^-3
- Cross entropy on raw logits

LAYER	SIZE	ACTIVATION
Input	784 (28x28 flattened)	—
Hidden 1	512	ReLU
Hidden 2	512	ReLU
Output	10	— (logits)

MNIST with an ANN: Results

We start with the PyTorch tutorial





METRIC	VALUE
Best test accuracy	97.4% (epoch 4)
Final test accuracy	95.5%
Best test loss	0.1029 (epoch 3)
Final test loss	0.1994
Trainable params	669706

97.4% best accuracy on 60,000 samples without using PyTorch Tutorial.

MNIST with an AI-SNN: Setup

Convert the ANN to spiking model

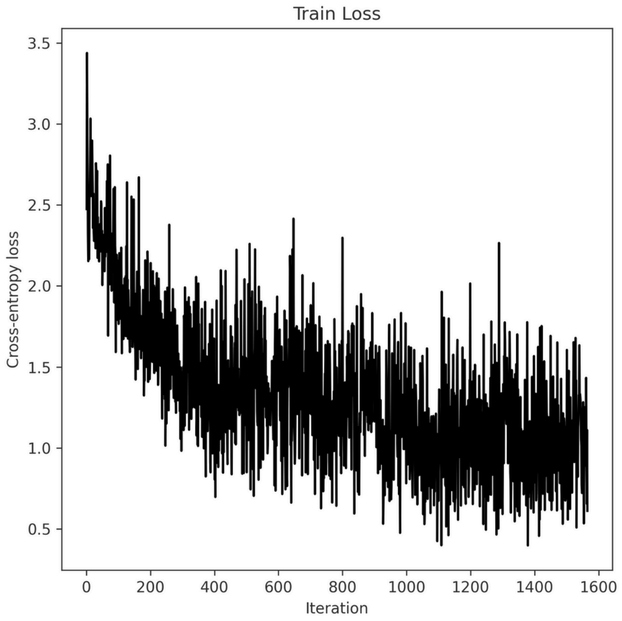
- **Constant-rate current injection**: the pixel values (scaled by input_scale) are injected as a steady excitatory conductance into the input neurons, held constant across all timesteps. ***NOT POISSON ENCODED FOR NOW.***
- **Surrogate gradient:** fast-sigmoid
- **Spike count decoding:** total number of spil $\tilde{\sigma}(x) = \frac{x}{2(1+|x|)^2}$ ch of the 10 output neurons over the full 100 ms window is take that class. The predicted digit is whichever output neuron fires the most.
- **Cross-entropy loss:** applied to these spike-count logits.
- **Weights:** E-E weights are clamped to be non-negative. Initial weights are drawn from normal and scaled by 1/sqrt(N)
- **Samples:** 5,000 not 60,000
- **No E-E recurrence**
- **Input to hidden & hidden to output layer weights are trained.**

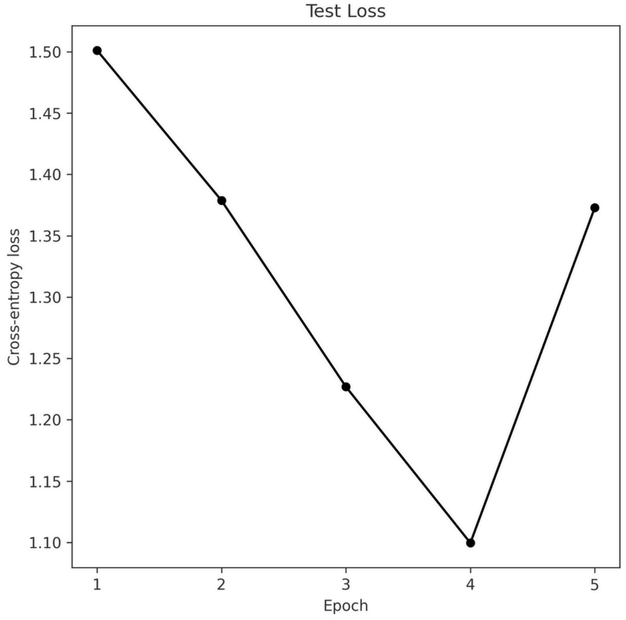
LAYER	POPULATION	SIZE	ROLE
Input	`E_in`	784	One neuron per pixel, driven by rate-coded image
Hidden	`E_hid`	64	Learned feature extraction
Output	`E_out`	10	One neuron per digit class

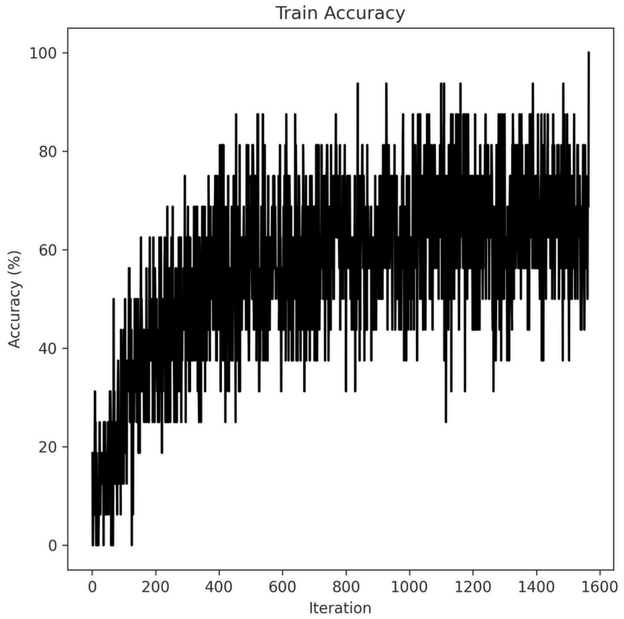
KEY	VALUE
`sim.neuron_model`	lif
`sim.dt_ms`	1
`sim.T_ms`	100
`meta.epochs`	5
`meta.batch_size`	16
`meta.lr`	0.001
`meta.input_scale`	3
`meta.subset_size`	5000
`constraints.nonnegative_weights`	true

MNIST with an standard-SNN: Results

Convert the ANN to spiking model



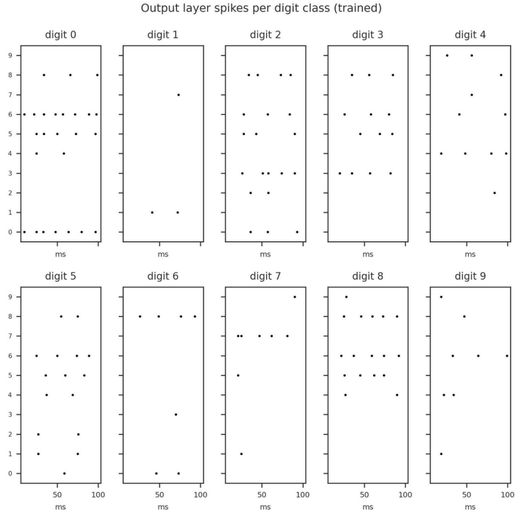


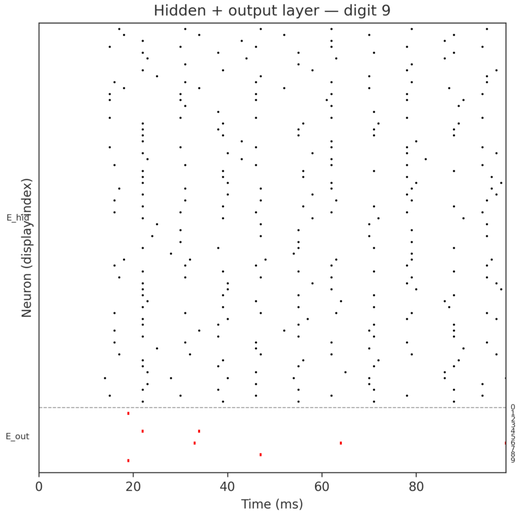


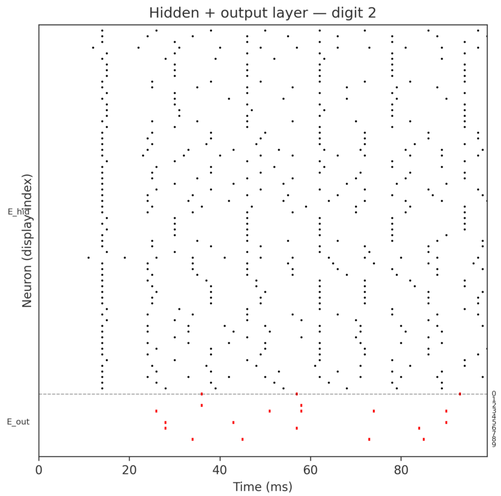
METRIC	VALUE
Best test accuracy	64.2% (epoch 4)
Final test accuracy	57.5%
Best test loss	1.0995 (epoch 4)
Final test loss	1.373
Trainable params	736164
Training time	340.3s

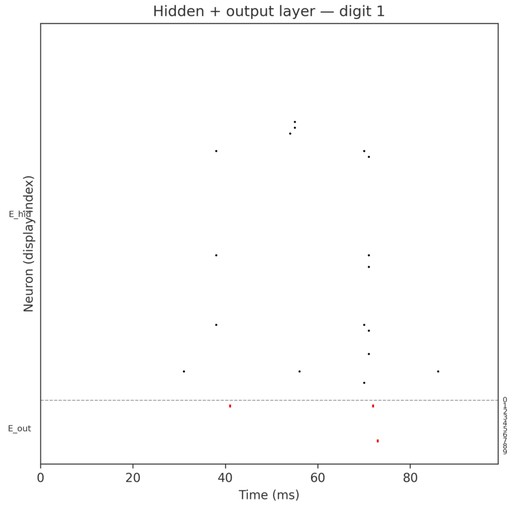
64.2% best accuracy on 5,000 samples without manual tuning.

MNIST with an standard-SNN: Rasters





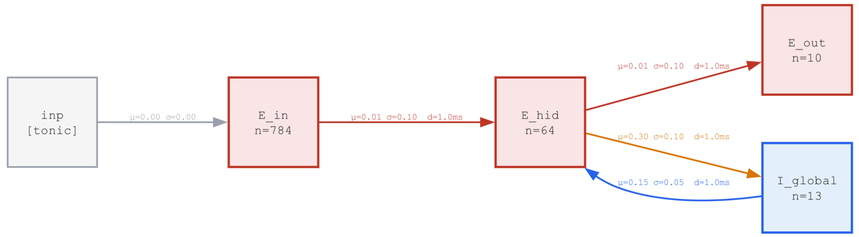




- **Oscillations in hidden layers**: quasi-periodic bursting at roughly 8–12 ms intervals across many digit classes.
- **Rhythm is not imposed:** input is constant-rate
- **Assumption:** arises from interplay of membrane time constants, refractory periods (t_ref=3ms), and the recurrent feedback through the synaptic conducance dynamics (t_ampa=2ms)
- **When rates are the same in output**: lowest digit chosen arbitrarily (super rough I know).

MNIST with an PING-SNN: Setup

INFERENCE-ONLY: reused weights from AI-SNN training



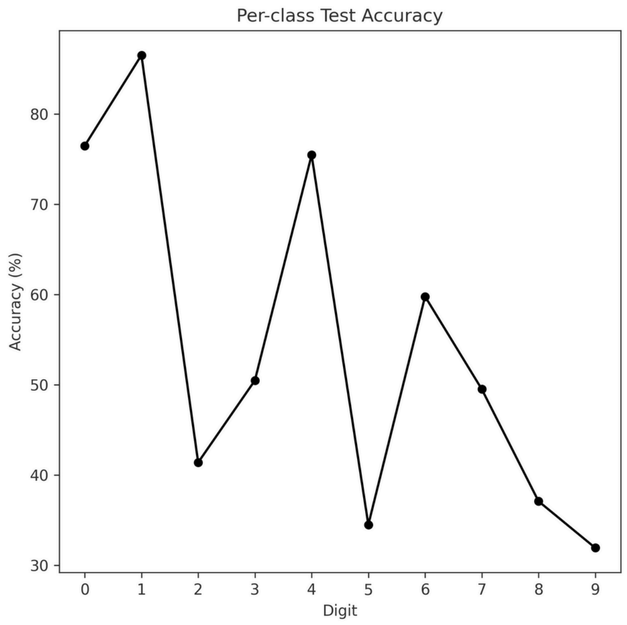
- **Identical setup to AI-SNN:** *except* hidden layer is connected to an I pool
- **E-I/I-E connections:** randomly initialized with weak normal distribution.

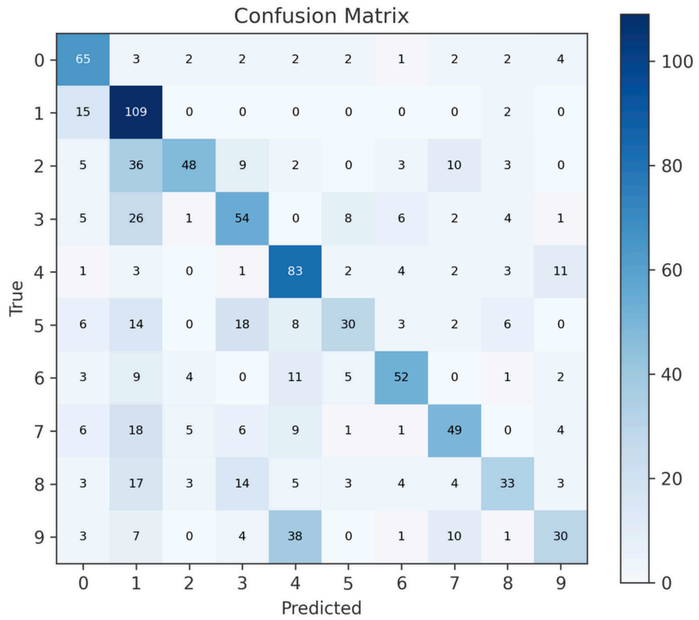
LAYER	POPULATION	SIZE	ROLE
Input	`E_in`	784	One neuron per pixel, driven by rate-coded image
Hidden	`E_hid`	64	Learned feature extraction
Output	`E_out`	10	One neuron per digit class

KEY	VALUE
`sim.neuron_model`	lif
`sim.dt_ms`	1
`sim.T_ms`	100
`meta.batch_size`	16
`meta.input_scale`	3
`meta.test_subset_size`	1000
`meta.weights_source`	study.8-ai-snn-ff

MNIST with an PING-SNN: Setup

INFERENCE-ONLY: reused weights from AI-SNN training



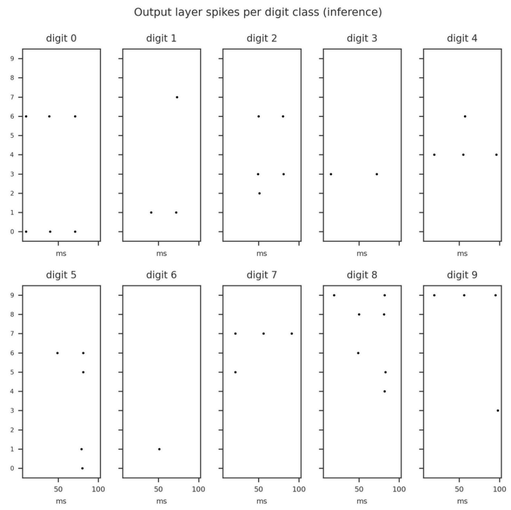


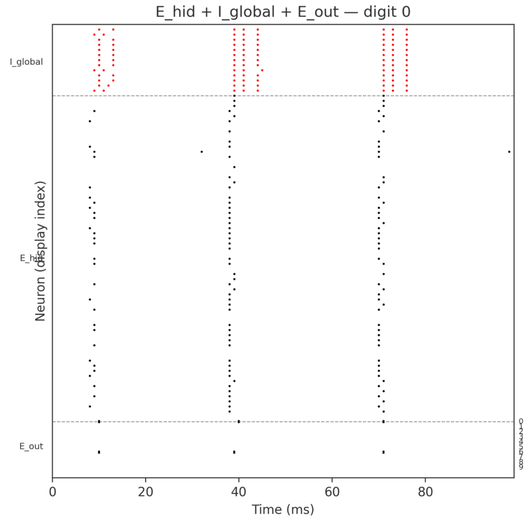
METRIC	VALUE
Test accuracy	55.3%
Test loss	1.3674
Test samples	1000
Weights source	study.8-ai-snn-ff
Inference time	11.3s

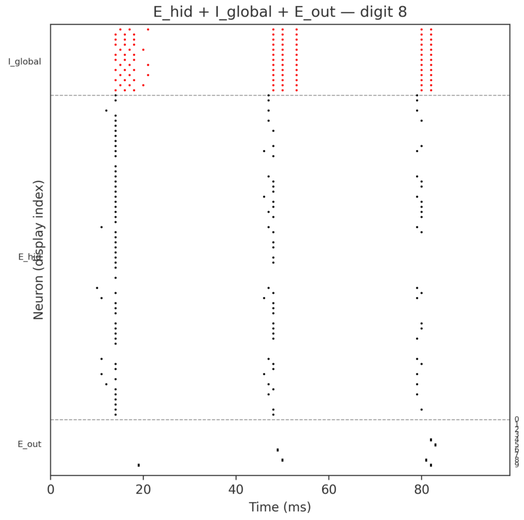
55.3% accuracy on 5,000 samples reusing weights from standard-SNN training.

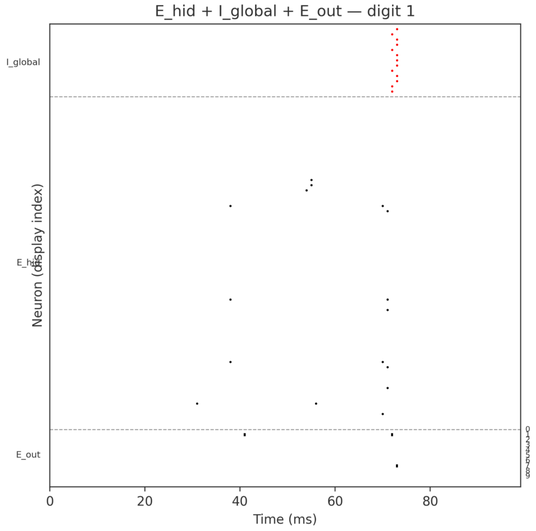
MNIST with an PING-SNN: Setup

INFERENCE-ONLY: reused weights from AI-SNN training









- **PING oscillations clearly visible in hidden layer**
- **Digit 1 is most sparse since it has the least dark pixels.**

Conclusion & Next steps.

- **Conclusion:** One days work, extremely rough but I think its a good result.
- **Conclusion:** I think we show the principle that the weights of a trained standard-SNN can be adopted by a PING-SNN with minor degradation of result.
- **Next step:** Poisson encoding not just pixel intensity to tonic input.
- **Next step:** Train on full MNIST dataset not just 5,000 samples.
- **Next step:** Play with the system and see if there are easy wins for improving accuracy.
- **Next step:** Investigate actually whats going on computation-wise.